

1 - Views, Controllers and Layouts

We setup our Laravel app and we learned how to take in requests through a route and send a response back. Now we are going to get into views, which are the pages and UI of our project. We will be using Blade templates for this as well as certain directives for dynamic data and things like conditionals and loops.

We'll also be working with controllers. We have been working directly in the routes closure, but usually your routes will pertain to a specific controller class and a specific method. So we will have a Job controller with a bunch of methods for different tasks. From our controllers we can load views and pass in data. We'll also look at using layouts with template inheritance and partials. So that we don't have to repeat ourselves and have things like the html, head and body tags in every one of our views.

We can also use Type Hinting in Laravel, so we will look at how that works as well.

2 - Create & Display Views

In Laravel, the view layer is responsible for handling the presentation of data to the user, typically in the form of HTML. Laravel's view system allows developers to separate the logic of the application from the display of the data, adhering to the Model-View-Controller (MVC) design pattern.

The view is where you define the user interface of your application. Laravel provides a powerful templating engine called Blade, which allows you to easily create reusable components, layouts, and dynamic content.

Using A Frontend Framework

Another very common use case for views is to use a frontend framework like Vue.js or React.js. You can use Laravel to serve the API and then use a frontend framework to consume the API and render the HTML. This is a very common pattern in modern web development. This makes your UI more interactive and responsive. Although you can certainly have interactive elements just by using either vanilla JavaScript or something like Alpine.js, which we're going to be using later.

Creating Views

To create a view in Laravel, you can use the `view()` helper function. The `view()` function takes the name of the view as its first argument. You can also pass data to the view but we'll look at that in the next lesson.

Ultimately, we will be using Blade templates, but views do not have to be Blade templates. They can be plain PHP files as well so that is what we will start with.

Let's create a new file at `resources/views/jobs.php` with the following content:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Jobs List</title>
```

```
</head>
<body>
  <h1>Available Jobs</h1>
  <ul>
    <li>Web Developer</li>
    <li>Software Engineer</li>
    <li>System Analyst</li>
    <li>Database Administrator</li>
  </ul>
</body>
</html>
```

It is just simple HTML for now. Of course, you can also use PHP in this file. You just need to use the `<?php ?>` tags. For example, you can replace the `h1` tag with the following PHP code:

```
<h1><?php echo 'Available Jobs' ?></h1>
```

Now, let's update our route to return this view:

```
Route::get('/jobs', function () {
    return view('jobs');
});
```

Now, when you visit `/jobs` in your browser, you should see the HTML.

Sub Folders For Views

You can also create subfolders in the `resources/views` directory to organize your views. For example, let's create a folder named `jobs` and rename the `jobs.php` file to `index.php` and move it to that folder.

Now, update the route to return the view from the subfolder:

```
Route::get('/jobs', function () {
    return view('jobs.index');
});
```

We can use dot notation to specify the subfolder and the view file. This is a common pattern in Laravel. So now, if we have let's say a view to add a new job, we can create a file named `create.php` in the `jobs` folder and return it like this:

```
Route::get('/jobs/create', function () {
    return view('jobs.create');
});
```

In the next lesson, I will show you how to pass data to views.

3 - Passing Data to Views

In the last lesson, we saw how to create a basic view. We haven't even gotten into Blade templates yet, but before we do that, I want to look at the different ways that you can pass data into a view. This is something that you will be doing a lot. For instance, if you fetch some data from your database through the model, you will want to pass that data to the view to display it.

This works the same way whether you are using Blade templates or plain PHP files. Let's look at how to pass data to a view using multiple methods.

Pass An Array

You can pass data to views by passing an associative array as the second argument to the `view()` function.

Let's pass in a title to the view:

```
Route::get('/jobs', function () {  
    return view('jobs', [  
        'title' => 'Available Jobs',  
    ]);  
});
```

Now, let's update our view to display the title:

```
<h1><?php echo $title; ?></h1>
```

->with() Method Alternative

You can also use the `->with()` method to pass data to views. The `->with()` method is a more fluent way of passing data to views. Here is an example:

```
Route::get('/jobs', function () {  
    return view('jobs')->with('title', 'Available Jobs');  
});
```

Passing an Array to Views

You can also pass arrays to views. Let's pass an array of jobs to the view:

```
Route::get('/jobs', function () {  
    $jobs = [  

```

```
        'Software Engineer',  
        'Web Developer',  
        'Data Scientist',  
    ];  
  
    return view('jobs', [  
        'title' => 'Available Jobs',  
        'jobs' => $jobs,  
    ]);  
});
```

You can also use the `->with()` method to pass an array to views. Here is an example:

```
Route::get('/jobs', function () {  
    $jobs = [  
        'Software Engineer',  
        'Web Developer',  
        'Data Scientist',  
    ];  
  
    return view('jobs')->with('title', 'Available Jobs')->with('jobs', $jobs);  
});
```

compact() Function

You can also use the `compact()` function to pass variables to views. The `compact()` function creates an array from the variable names you pass to it. Here is an example:

```
Route::get('/jobs', function () {  
    $title = 'Available Jobs';  
    $jobs = [  
        'Software Engineer',  
        'Web Developer',  
        'Data Scientist',  
    ];  
  
    return view('jobs', compact('title', 'jobs'));  
});
```

This is all preference. What I like to do is to use `->with` if there is only a single variable passed and `compact()` if there are multiple variables passed. Again, this is completely up to you and there is no right or wrong way to do it.

Displaying Data in Views

Now, let's update our view to display the jobs:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Jobs List</title>
  </head>

  <body>
    <h1><?php echo $title; ?></h1>
    <ul>
      <?php foreach ($jobs as $job) : ?>
        <li><?php echo htmlspecialchars($job, ENT_QUOTES, 'UTF-8'); ?></li>
      <?php endforeach; ?>
    </ul>
  </body>
</html>
```

As you can see, we have passed an array of jobs to the view and then looped through the jobs in the view to display them.

We used the `htmlspecialchars()` function to escape the output. This is to prevent XSS attacks. When we move to using Blade templates, we won't have to worry about escaping output, as Blade does this for us automatically. There are a lot of security features that not only Blade, but Laravel itself provides out of the box. We will look into these more later in the course.

In the next lesson, we will look into Blade templates and how to use them to render views.

4 - Blade Templates & Directives

Laravel uses a templating engine called Blade. Blade is a simple, yet powerful templating engine that allows you to write clean and concise templates. Blade templates are compiled into plain PHP code and cached until they are modified, meaning they are extremely fast. We can have dynamic content in our views by using Blade directives, which are special tags that start with `@`.

Creating a Blade Template

Let's take the `jobs/index.php` view we created in the previous chapter and convert it to a Blade template. To do this, rename the `jobs/index.php` file to `jobs/index.blade.php`. It will still work as you can use plain PHP in Blade templates. However, Blade provides additional features that make it easier to work with.

Outputting Variables

To output a variable in a Blade template, you can use double curly braces `{{ $variable }}`. Let's update our `jobs/index.blade.php` file to output the title using Blade syntax:

```
<h1>{{ $title }}</h1>
```

Now, when you visit `/jobs` in your browser, you should see the **Available Jobs** heading.

Directives

Blade provides a variety of directives that make it easy to work with data. Directives are prefixed with the `@` symbol. Let's look at a few of the most common directives.

@foreach Directive

Blade provides a `@foreach` directive that makes it easy to iterate over arrays. Let's update our `jobs/index.blade.php` file to display the list of jobs:

```
<ul>
  @foreach($jobs as $job)
    <li>{{ $job }}</li>
  @endforeach
</ul>
```

How much cleaner looking is that? Blade templates are a great way to keep your views clean and concise.

@if Directive

Blade also provides an `@if` directive that allows you to conditionally display content. Let's update our view to only display the list of jobs if there are jobs available:

```
@if(!empty($jobs))
<ul>
  @foreach($jobs as $job)
    <li>{{ $job }}</li>
  @endforeach
</ul>
@endif
```

Now if you clear the array in the route, the list of jobs will not be displayed. We could also use `count()` instead of `!empty()`.

@else Directive

Blade also provides an `@else` directive that allows you to display content when the condition is false. Let's update our view file to display a message when there are no jobs available:

```
@if(!empty($jobs))
<ul>
  @foreach($jobs as $job)
    <li>{{ $job }}</li>
  @endforeach
</ul>
```

```
@else
<p>No jobs available</p>
@endif
```

Now if you clear the array in the route, you will see the message **No jobs available**.

There are other directives that pertain to loops. We will look at some of those in the next lesson.

5 - More on Loops

There are some other directives that I want to show you that pertain to loops and iteration over arrays.

@forelse & @empty Directives

Blade provides a **@forelse** directive that gives us a better way of looping and having something else show if the array is empty. Let's update our view file to display a message when there are no Jobs Found using the **@forelse** directive:

```
<ul>
  @forelse($jobs as $job)
    <li>{{ $job }}</li>
  @empty
    <li>No Jobs Found</li>
  @endforelse
</ul>
```

I am going to use this instead of **@foreach** and **@if** because it is more readable and cleaner.

@break Directive

Blade provides a **@break** directive that allows you to break out of a loop. Let's update our view file to display a message when there are no Jobs Found using the **@break** directive:

```
<ul>
  @forelse($jobs as $job)
    @if($job === 'Web Developer')
      @break
    @endif
    <li>{{ $job }}</li>
  @empty
    <li>No Jobs Found</li>
  @endforelse
</ul>
```

Now it should not show any jobs after the Web Developer job.

@continue Directive

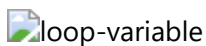
Blade provides a `@continue` directive that allows you to skip an iteration of a loop. Let's update our view file to display a message when there are no Jobs Found using the `@continue` directive:

```
<ul>
  @forelse($jobs as $job)
    @if($job === 'Web Developer')
      @continue
    @endif
    <li>{{ $job }}</li>
  @empty
    <li>No Jobs Found</li>
  @endforelse
</ul>
```

Now it should not show the Web Developer job.

\$loop Variable

Blade provides a `$loop` variable that allows you to access information about the current iteration of the loop. Here is a list of the available properties:



Let's try some of these out.

`$loop->index`

This will return the index of the current iteration of the loop. Let's update our view file to display the index of the current iteration of the loop:

```
<ul>
  @forelse($jobs as $job)
    <li>{{ $loop->index }} - {{ $job }}</li>
  @empty
    <li>No Jobs Found</li>
  @endforelse
</ul>
```

`$loop->iteration`

This will return the iteration of the current iteration of the loop. Let's update our view file to display the iteration of the current iteration of the loop:

```
<ul>
  @forelse($jobs as $job)
```

```
<li>{{ $loop->iteration }} - {{ $job }}</li>
@empty
<li>No Jobs Found</li>
@endforelse
</ul>
```

The difference between these two is that `iteration` will always start at 1, while `index` will start at 0.

`$loop->remaining`

This will return the remaining iterations of the loop. Let's update our view file to display the remaining iterations of the loop:

```
<ul>
@forelse($jobs as $job)
<li>{{ $loop->remaining }} - {{ $job }}</li>
@empty
<li>No Jobs Found</li>
@endforelse
</ul>
```

`$loop->count`

This will return the total number of iterations of the loop. Let's update our view file to display the total number of iterations of the loop:

```
<ul>
@forelse($jobs as $job)
<li>{{ $loop->count }} - {{ $job }}</li>
@empty
<li>No Jobs Found</li>
@endforelse
</ul>
```

`$loop->first`

This will return true if the current iteration of the loop is the first iteration of the loop. Let's update our view file to display a message when the current iteration of the loop is the first iteration of the loop:

```
<ul>
@forelse($jobs as $job)
@if($loop->first)
<li>First: {{ $job }}</li>
@else
<li>{{ $job }}</li>
@endif
@endforelse
</ul>
```

```
@empty
  <li>No Jobs Found</li>
@endforelse
</ul>
```

`$loop->last`

This will return true if the current iteration of the loop is the last iteration of the loop. Let's update our view file to display a message when the current iteration of the loop is the last iteration of the loop:

```
<ul>
  @forelse($jobs as $job)
    @if($loop->last)
      <li>Last: {{ $job }}</li>
    @else
      <li>{{ $job }}</li>
    @endif
  @empty
    <li>No Jobs Found</li>
  @endforelse
</ul>
```

`$loop->even`

This will return true if the current iteration of the loop is an even iteration of the loop. Let's update our view file to display a message when the current iteration of the loop is an even iteration of the loop:

```
<ul>
  @forelse($jobs as $job)
    @if($loop->even)
      <li>Even: {{ $job }}</li>
    @else
      <li>{{ $job }}</li>
    @endif
  @empty
    <li>No Jobs Found</li>
  @endforelse
</ul>
```

`$loop->odd`

This will return true if the current iteration of the loop is an odd iteration of the loop. Let's update our view file to display a message when the current iteration of the loop is an odd iteration of the loop:

```
<ul>
  @forelse($jobs as $job)
```

```

    @if($loop->odd)
      <li>Odd: {{ $job }}</li>
    @else
      <li>{{ $job }}</li>
    @endif
  @empty
    <li>No Jobs Found</li>
  @endforelse
</ul>

```

These can be very helpful when you are working with loops.

We don't need to use these now, so I am going to remove them.

Your final view for now, will look like this:

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Jobs List</title>
  </head>

  <body>
    <h1>{{ $title }}</h1>
    <ul>
      @forelse($jobs as $job)
        <li>{{ $job }}</li>
      @empty
        <li>No Jobs Found</li>
      @endforelse
    </ul>
  </body>
</html>

```

Now if you clear the array in the route, you will see the message **No Jobs Found**.

6 - Creating Controllers

So far, we have been working with routes and views. We have created routes to handle incoming requests and return views. However, as our application grows, we will need to handle more complex logic. This is where controllers come in.

What Is A Controller?

We talked about the role of controllers when we discussed the MVC design pattern, but just to touch on it a bit more, controllers are classes that group related request handling logic. They are responsible for processing

incoming requests, interacting with models, and returning views.

Controllers are stored in the `app/Http/Controllers` directory. You will see that there is a file called `Controller.php` in this directory. This file is the base controller class that all other controllers extend. This class does not contain any methods, but it provides a convenient location to add shared methods that all controllers can use. So every controller that we create will go in this folder and it will extend that base controller class.

Let's create our first controller. We can use the `make:controller` Artisan command to create a new controller. Run the following command in your terminal:

```
php artisan make:controller JobController
```

A Note On Naming Conventions

When creating controllers, it is common to use the singular form of the resource name followed by the word `Controller`. For example, if you are creating a controller to handle jobs, you would name it `JobController`. This is a convention that Laravel uses, but you are free to name your controllers however you like. [Here](#) is a list of Laravel naming conventions that you should follow.

This command will create a new file called `JobController.php` in the `app/Http/Controllers` directory. Open this file and you will see that it contains a class definition that extends the base controller class. It does not have any methods yet.

Adding Methods To The Controller

Let's add a method to our controller that will return a view. We will create a method called `index` that will return the `jobs/index.blade.php` view. Here is what the `JobController` class should look like:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class JobController extends Controller
{
    // Display a listing of the resource
    public function index()
    {
        return view('jobs.index');
    }
}
```

We are just returning the `jobs.index` view, just like we did in the route.

Using The Controller In A Route

Now that we have created our controller and added a method to it, we can use it in a route. Open the `routes/web.php` file and update the `/jobs` route to use the `JobController`.

First, we need to import the `JobController` at the top of the file:

```
use App\Http\Controllers\JobController;
```

Then, we can update the `/jobs` route to use the `index` method of the `JobController`:

```
Route::get('/jobs', [JobController::class, 'index']);
```

Here, we are using the `JobController` class and the `index` method as the callback for the route. This is a shorthand syntax for defining a controller action. The first parameter is the controller class, and the second parameter is the method that we want to call.

Now when you visit `/jobs` in your browser, you should see the jobs index view. However, if you have any variables, you will get an error.

Passing Data To Views

We can pass data the same way that we did from within the route. We can either pass an array, use the `with` method, or use the `compact` method. If it is more than one variable, I prefer to use the `compact` method. Here is how you can pass data to the view from the controller:

```
public function index()
{
    $title = 'Available Jobs';
    $jobs = [
        'Software Engineer',
        'Web Developer',
        'Data Scientist',
    ];

    return view('jobs/index', compact('title', 'jobs'));
}
```

Now the view will have access to the `$title` and `$jobs` variables. You can use these variables in the view the same way that you did before.

Home Controller & View

We have a job controller for the `/jobs` page. Let's create a home controller and view for the home page.

Create Home Controller

Use Artisan to create a new controller:

```
php artisan make:controller HomeController
```

Now we have a new controller at `app/Http/Controllers/HomeController.php`. Open the file and add an `index` method to the class that returns the `index` view:

```
public function index()  
{  
    return view('pages.index');  
}
```

This controller has a single method `index` that returns the `home` view in a `pages` folder.

Create Home View

Create a new view file at `resources/views/pages/index.blade.php`. If you still have the `welcome.blade.php` file, you can rename and move that and delete everything in it. Add the following code to the file:

```
<h1>Welcome to Workopia</h1>  
<p>Find your dream job today</p>
```

Update Routes

Open the `routes/web.php` file and DELETE the current homepage route and add the following code:

```
use App\Http\Controllers\HomeController;  
  
Route::get('/', [HomeController::class, 'index']);
```

Now when you visit the homepage, you should see the welcome message.

7 - Using Params, Request & Forms In A Controller

We already know how to work with params and the request object within a route. We can also use them in a controller. Let's see how we can do that. We are also going to create a form in a view and process the form in a controller.

Using Params In A Controller

We can use params in a controller the same way that we did in a route. We can type-hint the params in the method signature. Let's add a new method to our `JobController` that will take a param. We will create a method called `show` that will take a `jobId` param. Here is what the `JobController` class should look like:

```
class JobController extends Controller
{
    //...

    // Show job details
    public function show($id)
    {
        return "Showing job $id";
    }
}
```

Now, let's create a new route that uses the `show` method of the `JobController`. Open the `routes/web.php` file and create a new route that uses the `show` method of the `JobController`:

```
Route::get('/jobs/{id}', [JobController::class, 'show']);
```

You should see the string `Showing job {id}` when you visit `/jobs/1` in your browser. You can pass the `jobId` param to the `show` method and use it in the method.

Using The Request Object In A Controller

We can also use the request object in a controller. A common use of the request object is to get the input from a form. So let's create two methods in our `JobController` that will show a form and process the form. Here is what the `JobController` class should look like:

```
class JobController extends Controller
{
    //...

    // Show the form to create a job
    public function create()
    {
        return view('jobs.create');
    }

    // Store a job
    public function store(Request $request)
    {
        return $request->all();
    }
}
```

Create The Form View

Create a file at `resources/views/jobs/create.blade.php` and add the following code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Create Job</title>
  </head>

  <body>
    <h1>Create Job</h1>
    <form action="/jobs" method="POST">
      @csrf
      <input type="text" name="title" placeholder="Title" />
      <input type="text" name="description" placeholder="Description" />
      <button type="submit">Submit</button>
    </form>
  </body>
</html>
```

@csrf

You may have noticed the `@csrf` directive in the form. This is a security feature in Laravel to prevent cross-site request forgery (CSRF) attacks. It generates a hidden input field with a CSRF token that Laravel uses to verify the form submission. This will stop malicious users from submitting forms to your application without your knowledge. This is another reason why Laravel is so secure out of the box.

Using The Controller In A Route

Now that we have created our controller and added methods to it, we can use it in routes. Open the `routes/web.php` file and update the routes to use the `JobController`:

```
Route::get('/jobs/create', [JobController::class, 'create']);
Route::post('/jobs', [JobController::class, 'store']);
```

Make sure that you use the `post` method in the form and the route to process the form. Now when you visit `/jobs/create` in your browser, you should see the job form. When you submit the form, you should see the form data dumped to the screen.

You can also get individual form fields using the `input` method of the request object. Here is how you can get the `title` and `description` fields from the request object:

```
public function store(Request $request)
{
```

```
$title = $request->input('title');
$description = $request->input('description');

return "Title: $title, Description: $description";
}
```

Route Order

The order is important here because we have a route that uses the `show` method of the `JobController` that matches the `/jobs/{id}` route. If you put the `/jobs/{id}` route before the `/jobs/create` and `/jobs` routes, the `show` method will be called instead of the `create` and `store` methods. This is because Laravel will match the first route that it finds. So make sure that the `/jobs/{id}` route is at the bottom of the file.

Now go to `/jobs/create` in your browser and create a job. You should see the form data dumped to the screen. This will include the name, description and the CSRF token. You can use this data to save the job to the database or do whatever you want with it.

8 - Resource Routes

In Laravel, you can define a resource route that maps all the CRUD operations for a resource to controller methods. This is a convenient way to define routes for a resource without having to manually define each route, which is what we have been doing so far.

Resource Naming Convention

When creating a resource route, Laravel follows a naming convention for the controller methods.

Verb	URI Pattern	Controller Method	Description
GET	<code>/jobs</code>	<code>index</code>	Display a listing of the resource
GET	<code>/jobs/create</code>	<code>create</code>	Show the form for creating a new resource
POST	<code>/jobs</code>	<code>store</code>	Store a newly created resource in storage
GET	<code>/jobs/{job}</code>	<code>show</code>	Display the specified resource
GET	<code>/jobs/{job}/edit</code>	<code>edit</code>	Show the form for editing the specified resource
PUT/PATCH	<code>/jobs/{job}</code>	<code>update</code>	Update the specified resource in storage
DELETE	<code>/jobs/{job}</code>	<code>destroy</code>	Remove the specified resource from storage

We have been using the naming convention in our examples so far, but let's take a closer look at our routes:

php artisan route:list

```
GET|HEAD  /
.....
.....
```

```

GET|HEAD    jobs
.....
..... JobController@index
POST        jobs
.....
..... JobController@store
GET|HEAD    jobs/create
.....
.... JobController@create
GET|HEAD    jobs/{id}
.....
..... JobController@show
GET|HEAD    up
.....
.....

```

As you can see, we are following the convention.

Creating A Resource Route

We are now going to delete our entire `JobController` class and file and create a new one using the `--resource` flag. Delete the file and run the following command in your terminal:

```
php artisan make:controller JobController --resource
```

This will re-create a file named `JobController.php` in the `app/Http/Controllers` directory. Open this file and you will see that it contains a class definition that extends the base controller class. It also contains all the methods that we need for a resource route.

So any time that you want to create a resource route, you can use the `--resource` flag with the `make:controller` command.

Let's re-add the logic that we had previously. I also like to add a comment with a description and the route/method that the method is handling. Here is what the `JobController` class should look like so far:

```

<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class JobController extends Controller
{
    // @desc    Show all jobs
    // @route   GET /jobs
    public function index()
    {
        $title = 'Available Jobs';
        $jobs = [

```

```
        'Software Engineer',
        'Web Developer',
        'Data Scientist',
    ];

    return view('jobs/index', compact('title', 'jobs'));
}

// @desc Show create job form
// @route GET /jobs/create
public function create()
{
    return view('jobs.create');
}

// @desc Store a new job
// @route POST /jobs
public function store(Request $request)
{
    $title = $request->input('title');
    $description = $request->input('description');

    return "Title: $title, Description: $description";
}

// @desc Show a single job
// @route GET /jobs/{id}
public function show(string $id)
{
    return "Showing job $id";
}

// @desc Show the form for editing a job
// @route GET /jobs/{id}/edit
public function edit(string $id)
{
    return "Edit job $id";
}

// @desc Update a job
// @route PUT /jobs/{id}
public function update(Request $request, string $id)
{
    return "Update job $id";
}

// @desc Delete a job
// @route DELETE /jobs/{id}
public function destroy(string $id)
{
    return "Delete job $id";
}
}
```

We only have two methods that show a view. The rest are just returning strings. We will update all of this stuff soon.

Using The Resource Controller In A Route

Now that we have created our resource controller, we can use it in a route. Open the `routes/web.php` file and create a new resource route that uses the `JobController` controller:

```
Route::resource('jobs', JobController::class);
```

So we don't have to add any other routes for our jobs CRUD operations. This single line of code will create all the routes that we need for our jobs resource.

You can visit `/jobs`, `/jobs/create`, `/jobs/1`, `/jobs/1/edit`, etc. in your browser to see the results.

Adding New Routes To A Resource Controller

You can also add new routes to a resource controller. Let's say that you want to add a new route to show the form for creating a job. You can add a new route to the `JobController` class as you normally would. We aren't going to keep this method, but I just want to show you how to add a new method to a resource controller.

```
// @desc    Save a job to favorites
// @route   POST /jobs/{id}/save
public function save(Job $job): string
{
    return 'Save Job';
}
```

Then, you can add a new route to the `routes/web.php` file like this:

```
Route::get('/jobs/{id}/save', [JobController::class, 'save'])->name('jobs.save');
```

MAKE SURE that you add this BEFORE the `Route::resource('jobs', JobController::class);` line. If you add it after, it will not work. It will use the `show` method instead of the `save` method.

You can keep this route and method, but we won't use it until much later.

9 - Type Hinting

Type hinting is a way to specify the type of a variable in PHP. This is useful for both readability and maintainability of your code. It also helps to catch errors early on in the development process. In Laravel, this is completely optional, but it is a good practice to use it. We are using it in our arguments. We can also use it

with return types for our functions in the controller. Laravel has custom types that we can use. Here are some common ones:

- `Illuminate\Http\Request` - This is the request object that is passed to the controller method. It contains information about the HTTP request.
- `Illuminate\Http\Response` - This is the standard HTTP response object returned from a controller method. It can represent any kind of HTTP response.
- `Illuminate\Http\RedirectResponse` - This response type is used to redirect the user to another URL, often after a form submission.
- `Illuminate\Http\JsonResponse` - This is a response type specifically for returning JSON data, commonly used in APIs.
- `Illuminate\View\View` - This is the view object returned when rendering a Blade template, typically used in web applications.
- `Illuminate\Support\Collection` - This is a collection object returned from a controller method, often used to handle groups of models or data sets.
- `Illuminate\Auth\Access\Response` - This response type is used in authorization checks, allowing you to provide feedback on authorization decisions.
- `Illuminate\Pagination\LengthAwarePaginator` - This is a paginator object returned when paginating a collection of results, often used in index methods.
- `Symfony\Component\HttpFoundation\StreamedResponse` - This is used for streaming content, such as when generating large files for download.
- `Symfony\Component\HttpFoundation\BinaryFileResponse` - This is used for sending files to the user, typically used for file downloads.
- `Illuminate\Contracts\Routing\ResponseFactory` - This contract allows for creating various types of responses, like JSON, view, or download responses.

There are others for instance, the Eloquent ORM has it's own types.

Let's add return types to our `JobController` methods. Right now, a bunch of them are just returning strings. So those will be changed later.

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\View\View; // Add this line

class JobController extends Controller
{
    // @desc    Show all jobs
    // @route   GET /jobs
    public function index(): View
    {
        $title = 'Available Jobs';
        $jobs = [
            'Software Engineer',
            'Web Developer',
            'Data Scientist',
        ];
    }
}
```

```
        return view('jobs/index', compact('title', 'jobs'));
    }

    // @desc    Show create job form
    // @route    GET /jobs/create
    public function create(): View
    {
        return view('jobs.create');
    }

    // @desc    Store a new job
    // @route    POST /jobs
    public function store(Request $request): string
    {
        $title = $request->input('title');
        $description = $request->input('description');

        return "Title: $title, Description: $description";
    }

    // @desc    Show a single job
    // @route    GET /jobs/{id}
    public function show(string $id): string
    {
        return "Showing job $id";
    }

    // @desc    Show the form for editing a job
    // @route    GET /jobs/{id}/edit
    public function edit(string $id): string
    {
        return "Edit job $id";
    }

    // @desc    Update a job
    // @route    PUT /jobs/{id}
    public function update(Request $request, string $id): string
    {
        return "Update job $id";
    }

    // @desc    Delete a job
    // @route    DELETE /jobs/{id}
    public function destroy(string $id): string
    {
        return "Delete job $id";
    }
}
```

As you can see to use `View` we need to use `use Illuminate\View\View;` at the top of the file.

In the HomeController, let's add the `View` type hinting to the `index` method.

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Job;
use Illuminate\View\View;

class HomeController extends Controller
{
    public function index(): View
    {
        return view('pages.home');
    }
}
```

Again, this is optional but it does help with readability and maintainability of your code. I will be using it in the rest of the course.

10 - Layouts Using Template Inheritance

Right now, we have a couple views. We have a `jobs/index.blade.php` view and a `jobs/create.blade.php` view. Notice that both of these files have the HTML boilerplate with the `head` and `body` tags. If we have to add this boilerplate to every view, it will be a lot of duplication. This is where layouts come in.

A layout is a template that we can use to wrap around our views. We can define a layout that contains the HTML boilerplate and then include our views inside the layout. This way, we can avoid duplicating the boilerplate code in every view.

There are a few ways to use layouts. We can do it the traditional way using template Inheritance as well as using layouts using components, which is the more modern way. We're going to be using components, however I want to show you both ways because you probably will run into the old way using template inheritance and partials. Let's start with the that.

Create A Layout

Let's create a layout that we can use to wrap around our views. Create a file at `resources/views/layout.blade.php` and add the following code:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Workopia | Find and list jobs</title>
  </head>

  <body class="bg-gray-100">
    <h1>My App</h1>
    <main class="container mx-auto p-4 mt-4">@yield('content')</main>
```

```
</body>
</html>
```

The `@yield` directive is used to define a section that can be overridden by a child view. We will use this directive to include our view content inside the layout. I just put the `<h1>` there in the body so that you can see that this will show on every page that uses this layout. I also added some classes to the `body` and `main` tags for styling.

Use The Layout

Now that we have a layout, let's use it in our views. Open the `jobs/index.blade.php` file and replace the content with the following code:

```
@extends('layout') @section('content')
<h1>{{ $title }}</h1>

@forelse($jobs as $job)
<li>{{ $job }}</li>
@empty
<li>No Jobs Found</li>
@endforelse @endsection
```

Here, we are using the `@extends` directive to extend the layout. The argument we are passing in is the location and name of the layout relative to the views folder. If we had it in `resources/views/layouts/app.blade.php`, we would pass in `layouts.app`.

We are also using the `@section` directive to define the content that will be included in the layout. The `@section` directive takes a name as an argument. In this case, we are using the name `content`. That is what we passed into the main `@yield` directive in the layout.

Add Layout To Other Views

Do the same for the `pages/home.blade.php` file:

```
@extends('layout') @section('content')
<h1>Welcome to Workopia</h1>
<p>Find your dream job today</p>
@endsection
```

Do the same for the `jobs/create.blade.php` file:

```
@extends('layout') @section('content')
<h1>Create Job</h1>
<form action="/jobs" method="POST">
    @csrf
    <input type="text" name="title" placeholder="Title" />
```

```
<input type="text" name="description" placeholder="Description" />
<button type="submit">Submit</button>
</form>
@endsection
```

You should see the `<h1>My App</h1>` tag on all 3 pages. This is because we are extending the layout and defining the content that will be included in the layout using the `@section` directive. You can delete that `<h1>` tag from the layout now.

Title Section

Let's add another `@yield` directive for the title in the layout. Open the `layout.blade.php` file and update it like this:

```
<title>@yield('title', 'Workopia | Find or List a Job')</title>
```

Here we are using the `@yield` directive to define a default title for the layout. If a child view does not override this section, the default title will be used.

I want to use the default for `jobs/index.blade.php`, so we won't add a title section there, but for `jobs/create.blade.php`, add the following above the content section:

```
@section('title')
Create Job
@endsection
```

Now the title should change to `Create Job` when you visit the `/jobs/create` route.

11 - @include Directive & Partial Views

In the previous chapter, we learned how to use the `@extends` directive to create a layout. In this chapter, we will learn how to use the `@include` directive to include a partial view in another view. There is a more modern way to use layouts using components, but we will cover that in a later chapter. I want you to understand both ways.

Create A Partial View

Let's create a partial view that we can include in our views. Create a file at `resources/views/partials/navbar.blade.php` and add the following code:

```
<nav>
  <a href="/">Home</a>
  <a href="/jobs">Jobs</a>
```

```
<a href="/jobs/create">Create Job</a>
</nav>
```

Use The Partial View

Now that we have a partial view, let's include it in our layout. Open the `layout.blade.php` file and replace the `<h1>` tag with the following code:

```
@include('partials.navbar')
```

It should be right above the `<main>` tag. Now, the navbar will be included in every view that uses this layout.

Now you should see the menu on every page that uses this layout.

So you could proceed to build your website like this, but it is sort of the older way to do it. In the next section, we're really going to start on our project and we're going to use the more modern way of doing layouts using components.